Flower Classification

https://github.com/alyshwary/flower-classification-project

OVERVIEW

A CNN-based image classification project that identifies five flower types (Daisy, Dandelion, Rose, Sunflo
Recognition dataset and a custom TensorFlow/Keras model pipeline.

IMPACT

Built an end-to-end notebook workflow that prepares the dataset, trains a custom CNN, and evaluates mu
species from images.

TECHNICAL WALKTHROUGH

Used the Kaggle Flowers Recognition dataset with five classes: Daisy, Dandelion, Rose, Sunflower, an
class folders and loaded into the notebook for training and validation.

i) Install dependencies: TensorFlow, OpenCV, scikit-learn, matplotlib, seaborn.

ii) Place the dataset under flowers/<class_name>/ to match the expected directory structure.

Images are resized to a consistent shape, normalized to [0,1], and split into train/validation sets to ensu

i) Resize and normalize images to stabilize training.

ii) Create stratified train/validation splits for balanced class coverage.

Implemented a custom CNN in Keras with stacked convolution + pooling blocks followed by dense laye

i) Convolutional blocks learn spatial features of petals and textures.

ii) Dense layers map extracted features to 5-class softmax outputs.

Tracked training and validation curves to monitor convergence and prevent overfitting.

i) Visualized accuracy/loss trends to validate model stability.

ii) Evaluated class-wise performance using predictions on validation data.

The full workflow is implemented in the notebook for end-to-end reproducibility and easy experimentati

KEY DETAILS

1. Organized the Kaggle Flowers Recognition dataset into class-specific folders for five flower categori

2. Preprocessed images by resizing and normalizing pixel values, then created train/validation splits fo

3. Implemented a custom CNN architecture in TensorFlow/Keras for 5-class image classification.

4. Tracked training/validation accuracy and loss across epochs and visualized results with Matplotlib/S

5. Packaged the workflow into a reproducible Jupyter Notebook for easy experimentation and iteration.

6. Uses OpenCV for image loading/processing and scikit-learn for train/test splitting alongside TensorF

7. Serializes processed image data to a data.pickle file so subsequent runs skip re-reading raw images